

Angebots-KI

Der komplette Quellcode – Schritt für Schritt erklärt

Stand: 07.07.2026

Was macht dieses Programm?

Stell dir eine Vertriebsmitarbeiterin vor, die eine Kunden-E-Mail liest (»Wir brauchen 2 Pressen und 3 Kabelschneider ...«) und daraus ein ordentliches Angebots-PDF erstellt. Genau diese Arbeit übernimmt dieses Programm automatisch.

Es liest die E-Mail, sucht die passenden Produkte aus dem Firmenkatalog, formuliert die Texte, rechnet die Preise, prüft alles noch einmal selbst und gibt am Ende ein fertiges PDF aus. Für das »Verstehen« und »Formulieren« nutzt es eine künstliche Intelligenz (KI, hier »Claude«); für die Preise nutzt es feste Rechenregeln, damit die Zahlen immer stimmen.

Wie alles zusammenspielt (vereinfacht):

E-Mail + Katalog → KI liest → KI entwirft → Programm rechnet Preise → KI prüft selbst → PDF

Kleines Glossar (für Nicht-Techniker)

Python: Die Programmiersprache, in der alles geschrieben ist.

Bibliothek: Fertige Zusatzbausteine von anderen – z. B. »Streamlit« für die Oberfläche oder »ReportLab« für das PDF.

Funktion: Ein benannter Arbeitsschritt (»mache X«), den man wiederverwenden kann. Im Code am Wort def erkennbar.

KI / Claude: Die künstliche Intelligenz, die Texte versteht und schreibt.

API-Schlüssel: Eine Art Passwort, mit dem sich das Programm bei der KI anmeldet.

CSV: Eine einfache Tabellen-Datei (wie Excel) – hier der Produktkatalog.

PDF: Das fertige, druckbare Dokument.

Kommentare & Docstrings: Zeilen mit # und Texte in dreifachen Anführungszeichen (""") sind Erklärungen für Menschen und werden vom Computer nicht ausgeführt. Im Code unten sind sie farbig hervorgehoben.

app.py – Die Bedienoberfläche (das Schaufenster)

Dies ist der Teil, den man im Browser sieht und bedient. Er baut die Web-Seite auf: Felder zum Hochladen des Katalogs, ein Textfeld für die Kunden-E-Mail und den Knopf »Angebot erstellen«.

app.py nimmt die Eingaben entgegen, reicht sie an das Programm-Innere weiter und zeigt am Ende das Ergebnis samt PDF-Download an. Es enthält bewusst keine komplizierte Logik – es ist nur die Vermittlung zwischen Mensch und Maschine.

Die wichtigsten Teile

- **Seitenleiste** — Einstellungen wie API-Schlüssel, Steuersatz und Firmendaten.
- **_secret / _check_password** — Lesen des geheimen Schlüssels in der Cloud und ein optionaler Passwortschutz für die öffentliche Demo.
- **Datei-Uploads & Textfeld** — Katalog (CSV), Vorlage (PDF) und die Kundenanfrage.
- **Knopf »Angebot erstellen«** — startet den Ablauf und zeigt einen Fortschritt an.
- **render_result** — zeigt Positionen, Summen, Hinweise und den PDF-Download.

Quellcode: app.py

```
1  """Streamlit-Oberfläche der Angebots-KI.
2
3  Start: streamlit run app.py
4  """
5
6  from __future__ import annotations
7
8  import io
9  import os
10 from pathlib import Path
11
12 import streamlit as st
13 from dotenv import load_dotenv
14
15 from angebots_ki.catalog import Catalog
16 from angebots_ki.claude_client import AngebotsKI
17 from angebots_ki.config import CompanyInfo, Settings
18 from angebots_ki.errors import OfferGenerationError
19 from angebots_ki.pipeline import generate_offer
20 from angebots_ki.utils import extract_pdf_text, fmt_money, fmt_qty
21 from angebots_ki import storage
22
23 load_dotenv()
24
25
26 def _secret(name: str, default: str = "") -> str:
27     """Liest einen Wert aus st.secrets, ohne lokal (ohne secrets.toml) zu crashen."""
28     try:
29         return str(st.secrets[name])
30     except Exception:
31         return default
32
33
34 # Auf Streamlit Community Cloud wird der API-Key als Secret hinterlegt. Hier in die
35 # Umgebung spiegeln, damit der bestehende Key-Pfad (os.getenv / anthropic) unverändert
36 # funktioniert. Lokal (ohne Secret) bleibt alles wie gehabt (.env oder Seitenleiste).
37 if not os.getenv("ANTHROPIC_API_KEY"):
38     _cloud_key = _secret("ANTHROPIC_API_KEY")
39     if _cloud_key:
40         os.environ["ANTHROPIC_API_KEY"] = _cloud_key
41
42 # AWS-Zugangsdaten & S3-Konfiguration aus den Secrets in die Umgebung spiegeln
43 # (optional – ohne diese Werte bleibt die S3-Quelle einfach ausgeblendet).
44 for _aws_name in ("AWS_ACCESS_KEY_ID", "AWS_SECRET_ACCESS_KEY", "AWS_DEFAULT_REGION",
45                  "S3_BUCKET", "S3_CATALOG_PREFIX", "S3_TEMPLATE_PREFIX"):
46     if not os.getenv(_aws_name):
```

```

47     _aws_val = _secret(_aws_name)
48     if _aws_val:
49         os.environ[_aws_name] = _aws_val
50
51 SAMPLE_DIR = Path(__file__).parent / "sample_data"
52
53 st.set_page_config(page_title="Angebots-KI", page_icon="■", layout="wide")
54
55
56 def _check_password() -> None:
57     """Optionaler Passwortschutz - nur aktiv, wenn das Secret APP_PASSWORD gesetzt ist.
58
59     Schützt die öffentliche Demo-URL davor, dass Fremde dein API-Guthaben verbrauchen.
60     Ohne gesetztes Secret (z. B. lokal) ist die App frei zugänglich.
61     """
62     expected = _secret("APP_PASSWORD")
63     if not expected or st.session_state.get("auth_ok"):
64         return
65     st.title("■ Angebots-KI")
66     st.caption("Bitte Passwort eingeben, um die Demo zu öffnen.")
67     pw = st.text_input("Passwort", type="password")
68     if pw == expected:
69         st.session_state["auth_ok"] = True
70         st.rerun()
71     elif pw:
72         st.error("Falsches Passwort.")
73     st.stop()
74
75
76 _check_password()
77
78
79 # -----
80 # Hilfsfunktionen
81 # -----
82 def _read_sample(name: str) -> str:
83     path = SAMPLE_DIR / name
84     if path.exists():
85         return path.read_text(encoding="utf-8")
86     return ""
87
88
89 @st.cache_data(show_spinner=False, ttl=120)
90 def _s3_list(bucket: str, prefix: str, extensions: tuple) -> list:
91     # bucket ist nur Teil des Cache-Schlüssels; S3Store liest ihn aus der Umgebung.
92     return storage.S3Store().list_files(prefix, extensions)
93
94
95 @st.cache_data(show_spinner=False, ttl=300)
96 def _s3_bytes(bucket: str, key: str) -> bytes:
97     return storage.S3Store().get_bytes(key)
98
99
100 def _resolve_catalog() -> Catalog | None:
101     """Zeigt die Quellenauswahl für den Katalog und liefert das geladene Catalog-Objekt."""
102     sources = (["AWS S3"] if storage.is_configured() else []) + ["Datei hochladen", "Beispielkatalog"]
103     src = st.radio("Katalog-Quelle", sources, horizontal=True, key="cat_src",
104                   index=sources.index("Beispielkatalog"))
105     catalog: Catalog | None = None
106     try:
107         if src == "AWS S3":
108             bucket = os.getenv("S3_BUCKET", "")
109             keys = _s3_list(bucket, storage.catalog_prefix(), (".csv",))
110             if keys:
111                 sel = st.selectbox("Katalog in AWS S3", keys, key="cat_s3_key")
112                 catalog = Catalog.load(io.BytesIO(_s3_bytes(bucket, sel)))
113             else:
114                 st.info("Keine CSV-Dateien im Katalog-Ordner des Buckets gefunden.")
115         elif src == "Datei hochladen":
116             up = st.file_uploader("Produktkatalog (CSV)", type=["csv"], key="catalog")
117             if up is not None:

```

```

118         catalog = Catalog.load(io.BytesIO(up.getvalue()))
119     elif (SAMPLE_DIR / "produktkatalog.csv").exists():
120         catalog = Catalog.load(SAMPLE_DIR / "produktkatalog.csv")
121 except OfferGenerationError as exc:
122     st.error(str(exc))
123
124 if catalog is not None:
125     with st.expander(f"Katalogvorschau ({len(catalog)} Artikel)", expanded=False):
126         st.caption(
127             f"Erkannt - Artikelnummer: '{catalog.col_sku}', Name: '{catalog.col_name}', "
128             f"Preis: '{catalog.col_price}'"
129             + (f", Einheit: '{catalog.col_unit}'" if catalog.col_unit else "")
130             + (f", {len(catalog.spec_cols)} Spezifikationsspalten" if catalog.spec_cols else "")
131         )
132         st.dataframe(catalog.preview_df(), use_container_width=True, hide_index=True)
133 return catalog
134
135
136 def _resolve_template() -> str:
137     """Zeigt die Quellenauswahl für die Vorlage und liefert den extrahierten Text."""
138     sources = (["AWS S3"] if storage.is_configured() else []) + ["Datei hochladen", "Keine"]
139     src = st.radio("Vorlage-Quelle", sources, horizontal=True, key="tpl_src",
140                   index=sources.index("Keine"))
141     text = ""
142     try:
143         if src == "AWS S3":
144             bucket = os.getenv("S3_BUCKET", "")
145             keys = _s3_list(bucket, storage.template_prefix(), (".pdf",))
146             if keys:
147                 sel = st.selectbox("Vorlage in AWS S3", keys, key="tpl_s3_key")
148                 text = extract_pdf_text(io.BytesIO(_s3_bytes(bucket, sel)))
149                 st.caption(f"■ Vorlage gelesen: {len(text)} Zeichen")
150             else:
151                 st.info("Keine PDF-Vorlagen im Vorlagen-Ordner des Buckets gefunden.")
152         elif src == "Datei hochladen":
153             up = st.file_uploader("Vorlage (PDF)", type=["pdf"], key="template")
154             if up is not None:
155                 text = extract_pdf_text(io.BytesIO(up.getvalue()))
156                 st.caption(f"■ Vorlage gelesen: {len(text)} Zeichen")
157     except OfferGenerationError as exc:
158         st.error(str(exc))
159     return text
160
161
162 # -----
163 # Sidebar: Einstellungen
164 # -----
165 st.sidebar.header("■ Einstellungen")
166
167 env_key = os.getenv("ANTHROPIC_API_KEY", "")
168 api_key_input = st.sidebar.text_input(
169     "Anthropic API-Key",
170     type="password",
171     value="",
172     help="Leer lassen, wenn ANTHROPIC_API_KEY als Umgebungsvariable/.env gesetzt ist.",
173 )
174 if env_key and not api_key_input:
175     st.sidebar.caption("■ API-Key aus der Umgebung wird verwendet.")
176 elif not env_key and not api_key_input:
177     st.sidebar.warning("Kein API-Key gefunden - bitte oben eingeben oder .env setzen.")
178
179 with st.sidebar.expander("Modell & Steuer", expanded=False):
180     model = st.text_input("Modell", value=Settings.model)
181     currency = st.selectbox("Währung", ["EUR", "USD", "GBP", "CHF"], index=0)
182     vat_pct = st.number_input("MwSt-Satz (%)", min_value=0.0, max_value=30.0,
183                              value=19.0, step=0.5)
184     validity = st.number_input("Gültigkeit (Tage)", min_value=1, max_value=365, value=30)
185     review_iters = st.number_input("Max. Prüf-/Korrekturdurchläufe", min_value=1,
186                                    max_value=3, value=1)
187     use_thinking = st.checkbox("Adaptives Thinking nutzen", value=True)
188

```

```

189 with st.sidebar.expander("Eigene Firmendaten (Briefkopf)", expanded=False):
190     c_name = st.text_input("Firmenname", value=CompanyInfo.name)
191     c_addr = st.text_area("Adresse (eine Zeile pro Feld)",
192                          value="\n".join(CompanyInfo().address_lines), height=90)
193     c_phone = st.text_input("Telefon", value=CompanyInfo.phone)
194     c_email = st.text_input("E-Mail", value=CompanyInfo.email)
195     c_web = st.text_input("Website", value=CompanyInfo.website)
196     c_tax = st.text_input("Steuernummer/USt-IdNr.", value=CompanyInfo.tax_id)
197     c_bank = st.text_input("Bankverbindung", value=CompanyInfo().bank)
198
199 with st.sidebar.expander("AWS S3 (Produktdaten & Vorlagen)", expanded=False):
200     _s3_bucket_in = st.text_input("Bucket-Name", value=os.getenv("S3_BUCKET", ""))
201     _s3_region_in = st.text_input("Region",
202                                   value=os.getenv("AWS_DEFAULT_REGION", "") or "eu-central-1")
203     _s3_key_in = st.text_input("AWS Access Key ID", value="", type="password")
204     _s3_secret_in = st.text_input("AWS Secret Access Key", value="", type="password")
205     if _s3_bucket_in:
206         os.environ["S3_BUCKET"] = _s3_bucket_in.strip()
207     if _s3_region_in:
208         os.environ["AWS_DEFAULT_REGION"] = _s3_region_in.strip()
209     if _s3_key_in:
210         os.environ["AWS_ACCESS_KEY_ID"] = _s3_key_in.strip()
211     if _s3_secret_in:
212         os.environ["AWS_SECRET_ACCESS_KEY"] = _s3_secret_in.strip()
213     st.caption("■ Mit S3 verbunden" if storage.is_configured()
214              else "Nicht konfiguriert (optional – Upload/Beispiel funktionieren weiter). "
215              "Anleitung: AWS_SETUP.md")
216
217
218 def build_settings() -> Settings:
219     company = CompanyInfo(
220         name=c_name.strip() or "Unternehmen",
221         address_lines=[ln.strip() for ln in c_addr.splitlines() if ln.strip()],
222         phone=c_phone.strip(), email=c_email.strip(), website=c_web.strip(),
223         tax_id=c_tax.strip(), bank=c_bank.strip(),
224     )
225     return Settings(
226         model=model.strip() or Settings.model,
227         currency=currency,
228         vat_rate=vat_pct / 100.0,
229         default_validity_days=int(validity),
230         max_review_iterations=int(review_iters),
231         use_thinking=use_thinking,
232         company=company,
233     )
234
235
236 # -----
237 # Hauptbereich: Eingaben
238 # -----
239 st.title("■ Angebots-KI")
240 st.caption(
241     "Liest eine Kunden-E-Mail, erstellt mit Claude ein Angebot anhand deines "
242     "Produktkatalogs und einer Beispiel-Vorlage, prüft es selbst und gibt ein PDF aus."
243 )
244
245 left, right = st.columns(2)
246
247 with left:
248     st.subheader("1 · Produktkatalog")
249     catalog = _resolve_catalog()
250
251     st.subheader("2 · Beispiel-Angebot als Vorlage (optional)")
252     template_text = _resolve_template()
253
254 with right:
255     st.subheader("3 · Kundenanfrage (E-Mail)")
256     email_upload = st.file_uploader("E-Mail-Datei (.txt/.eml, optional)",
257                                     type=["txt", "eml"], key="email")
258     if st.button("■ Beispiel-Anfrage laden"):
259         st.session_state["email_text"] = _read_sample("beispiel_email.txt")

```

```

260
261 if email_upload is not None and "email_loaded" not in st.session_state:
262     st.session_state["email_text"] = email_upload.getvalue().decode(
263         "utf-8", errors="replace")
264     st.session_state["email_loaded"] = True
265
266 email_text = st.text_area(
267     "Text der Angebotsanfrage",
268     key="email_text",
269     height=260,
270     placeholder="Sehr geehrte Damen und Herren, wir benötigen ein Angebot über ...",
271 )
272
273 st.divider()
274 generate = st.button("■ Angebot erstellen", type="primary", use_container_width=True)
275
276
277 # -----
278 # Generierung
279 # -----
280 def render_result(result) -> None:
281     final = result.final
282     st.success(f"Angebot **{final.offer_number}** erstellt.")
283
284     # Download oben anbieten
285     st.download_button(
286         "■ Angebot als PDF herunterladen",
287         data=result.pdf_bytes,
288         file_name=f"Angebot_{final.offer_number}.pdf",
289         mime="application/pdf",
290         use_container_width=True,
291     )
292
293     if final.warnings:
294         st.warning("Hinweise zu Positionen:\n\n- " + "\n- ".join(final.warnings))
295     if final.review_issues:
296         st.info("Anmerkungen aus der KI-Selbstprüfung:\n\n- "
297             + "\n- ".join(final.review_issues))
298
299     tab_pos, tab_text, tab_analysis = st.tabs(
300         ["Positionen & Summen", "Texte", "Erkannte Anfrage"])
301
302     with tab_pos:
303         rows = [
304             {
305                 "Pos.": it.position,
306                 "Art.-Nr.": it.sku,
307                 "Bezeichnung": it.name,
308                 "Menge": f"{fmt_qty(it.quantity)} {it.unit}",
309                 "Einzelpreis": fmt_money(it.unit_price, final.currency),
310                 "Gesamt": fmt_money(it.line_total, final.currency),
311             }
312             for it in final.line_items
313         ]
314         st.dataframe(rows, use_container_width=True, hide_index=True)
315         c1, c2, c3 = st.columns(3)
316         c1.metric("Zwischensumme", fmt_money(final.subtotal, final.currency))
317         c2.metric(f"MwSt ({final.vat_rate * 100:.0f}%)",
318             fmt_money(final.vat_amount, final.currency))
319         c3.metric("Gesamtbetrag", fmt_money(final.total, final.currency))
320
321     with tab_text:
322         st.markdown(f"***Titel:** {final.title}")
323         st.markdown("***Einleitung***")
324         st.write(final.intro_text)
325         st.markdown("***Abschluss***")
326         st.write(final.closing_text)
327         st.markdown(f"***Zahlung:** {final.payment_terms}")
328         st.markdown(f"***Lieferung/Leistung:** {final.delivery_terms}")
329
330     with tab_analysis:

```

```

331     cust = result.analysis.customer
332     st.markdown(
333         f"Kunde: {cust.company or '-'} · {cust.contact_name or '-'} · "
334         f"{cust.email or '-'}"
335     )
336     if result.analysis.summary:
337         st.markdown(f"Zusammenfassung: {result.analysis.summary}")
338     st.markdown("Angefragte Positionen:")
339     st.dataframe(
340         [
341             {"Beschreibung": i.description, "Menge": fmt_qty(i.quantity),
342              "Einheit": i.unit, "Hinweis": i.notes}
343             for i in result.analysis.items
344         ],
345         use_container_width=True, hide_index=True,
346     )
347     if result.analysis.open_questions:
348         st.markdown("Offene Punkte:\n- "
349                     + "\n- ".join(result.analysis.open_questions))
350
351
352 if generate:
353     if catalog is None:
354         st.error("Bitte zuerst einen Produktkatalog (CSV) bereitstellen.")
355     elif not (email_text or "").strip():
356         st.error("Bitte den Text der Kundenanfrage einfügen.")
357     else:
358         settings = build_settings()
359         try:
360             ki = AngebotsKI(settings, api_key=api_key_input or None)
361         except Exception as exc: # z. B. fehlender Key
362             st.error(f"KI konnte nicht initialisiert werden: {exc}")
363         else:
364             status = st.status("Starte Angebotserstellung ...", expanded=True)
365             try:
366                 result = generate_offer(
367                     email_text=email_text,
368                     catalog=catalog,
369                     template_text=template_text,
370                     ki=ki,
371                     settings=settings,
372                     progress=lambda m: status.write(m),
373                 )
374                 status.update(label="Angebot fertig erstellt ■", state="complete")
375                 st.session_state["result"] = result
376             except OfferGenerationError as exc:
377                 status.update(label="Abgebrochen", state="error")
378                 st.error(str(exc))
379             except Exception as exc: # unerwartete Fehler
380                 status.update(label="Unerwarteter Fehler", state="error")
381                 st.exception(exc)
382
383 if "result" in st.session_state:
384     st.divider()
385     render_result(st.session_state["result"])
386

```


pipeline.py – Der Ablaufplan (der Dirigent)

Diese Datei bestimmt die Reihenfolge der Arbeitsschritte – wie ein Dirigent, der den Musikern nacheinander den Einsatz gibt.

Sie ruft der Reihe nach auf: Anfrage verstehen → Angebot entwerfen → Preise berechnen → Angebot selbst prüfen → PDF erzeugen.

Die wichtigsten Teile

- **generate_offer** — führt die fünf Schritte in der richtigen Reihenfolge aus.
- **progress** — meldet Zwischenstände (»Analysiere ...«) an die Oberfläche.
- **OfferResult** — bündelt das Endergebnis (Angebot + fertiges PDF).

Quellcode: `angebots_ki/pipeline.py`

```
1  """Orchestrierung: E-Mail -> Analyse -> Entwurf -> Preise -> Selbstprüfung -> PDF."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from typing import Callable, Optional
7
8  from .catalog import Catalog
9  from .claude_client import AngebotsKI
10 from .config import Settings
11 from .models import FinalOffer, OfferDraft, RequestAnalysis
12 from .pdf_builder import build_offer_pdf
13 from .pricing import assemble_final, price_offer, render_priced_for_review
14
15 ProgressCallback = Optional[Callable[[str], None]]
16
17
18 @dataclass
19 class OfferResult:
20     final: FinalOffer
21     pdf_bytes: bytes
22     analysis: RequestAnalysis
23     draft: OfferDraft
24
25
26 def generate_offer(
27     *,
28     email_text: str,
29     catalog: Catalog,
30     template_text: str,
31     ki: AngebotsKI,
32     settings: Settings,
33     progress: ProgressCallback = None,
34 ) -> OfferResult:
35     """Führt den kompletten Ablauf aus und liefert Angebot + PDF.
36
37     ``progress`` wird (falls gesetzt) mit kurzen Statusmeldungen aufgerufen.
38     """
39
40     def step(msg: str) -> None:
41         if progress:
42             progress(msg)
43
44     if not email_text.strip():
45         from .errors import OfferGenerationError
46
47         raise OfferGenerationError("Bitte zuerst den Text der Kundenanfrage einfügen.")
48
49     step("Analysiere die Kundenanfrage ...")
50     analysis = ki.analyze_request(email_text, catalog)
51
```

```

52     step("Erstelle den Angebotsentwurf ...")
53     draft = ki.draft_offer(analysis, catalog, template_text)
54     priced = price_offer(draft, catalog, settings)
55
56     review_issues: list[str] = []
57     iterations = max(1, settings.max_review_iterations)
58     for n in range(iterations):
59         step(f"Prüfe Text und Daten (Durchlauf {n + 1}/{iterations}) ...")
60         review_text = render_priced_for_review(draft, priced, analysis, settings)
61         review = ki.review_offer(review_text, analysis, catalog, template_text)
62         review_issues = review.issues
63         # Verbesserte Fassung übernehmen; Preise bleiben durch erneutes Pricing autoritativ.
64         draft = review.corrected_offer
65         priced = price_offer(draft, catalog, settings)
66         if review.approved:
67             break
68
69     step("Erzeuge das PDF ...")
70     final = assemble_final(analysis, draft, priced, review_issues, settings)
71     pdf_bytes = build_offer_pdf(final, settings)
72
73     step("Fertig.")
74     return OfferResult(final=final, pdf_bytes=pdf_bytes, analysis=analysis, draft=draft)
75

```

claude_client.py – Das Gehirn: die Verbindung zur KI

Hier spricht das Programm mit der künstlichen Intelligenz (Claude). Drei Aufgaben gehen an die KI: (1) die Kunden-E-Mail lesen und verstehen, (2) ein Angebot entwerfen, (3) das eigene Angebot kritisch prüfen.

Wichtig: Die KI darf Texte schreiben und Artikel auswählen, aber KEINE Preise erfinden – die kommen später aus dem Katalog. Die langen Texte in Anführungszeichen sind die »Arbeitsanweisungen« an die KI.

Die wichtigsten Teile

- **analyze_request** — zieht Kundendaten und gewünschte Positionen aus der E-Mail.
- **draft_offer** — wählt passende Artikel (per Artikelnummer) und formuliert die Texte.
- **review_offer** — prüft: Ist alles enthalten? Passen die Artikel? Stimmt der Text?
- **_structured** — sorgt dafür, dass die KI-Antwort immer eine feste, weiterverarbeitbare Form hat.

Quellcode: `angebots_ki/claude_client.py`

```
1  """Anbindung an Claude: Anfrage analysieren, Angebot entwerfen, selbst prüfen."""
2
3  from __future__ import annotations
4
5  from typing import Any, Optional, Type, TypeVar
6
7  import anthropic
8  from pydantic import BaseModel
9
10 from .catalog import Catalog
11 from .config import Settings
12 from .errors import OfferGenerationError
13 from .models import OfferDraft, RequestAnalysis, ReviewResult
14
15 T = TypeVar("T", bound=BaseModel)
16
17
18 def _strictify(node: Any) -> Any:
19     """Macht ein Pydantic-JSON-Schema kompatibel zu strukturierten Outputs.
20
21     Wird nur im Fallback (ältere SDKs ohne ``messages.parse``) benötigt: setzt auf
22     jedem Objekt ``additionalProperties: false`` und ``required`` = alle Felder.
23     """
24     if isinstance(node, dict):
25         node.pop("title", None)
26         if node.get("type") == "object" and "properties" in node:
27             node["additionalProperties"] = False
28             node["required"] = list(node["properties"].keys())
29         return {k: _strictify(v) for k, v in node.items()}
30     if isinstance(node, list):
31         return [_strictify(v) for v in node]
32     return node
33
34
35 class AngebotsKI:
36     """Kapselt alle KI-Aufrufe für die Angebotserstellung."""
37
38     def __init__(self, settings: Settings, api_key: Optional[str] = None):
39         self.settings = settings
40         # Ohne expliziten Key liest das SDK ANTHROPIC_API_KEY aus der Umgebung.
41         self.client = (
42             anthropic.Anthropic(api_key=api_key) if api_key else anthropic.Anthropic()
43         )
44
45     # -- generischer strukturierter Aufruf -----
46
47     def _structured(
48         self,
49         *,
50         system: str,
```

```

51     user: str,
52     schema: Type[T],
53     thinking: bool = True,
54     max_tokens: int = 16000,
55 ) -> T:
56     base: dict[str, Any] = dict(
57         model=self.settings.model,
58         max_tokens=max_tokens,
59         system=system,
60         messages=[{"role": "user", "content": user}],
61     )
62     if thinking and self.settings.use_thinking:
63         base["thinking"] = {"type": "adaptive"}
64
65     # Bevorzugt: messages.parse() validiert direkt gegen das Pydantic-Modell.
66     parse = getattr(self.client.messages, "parse", None)
67     if parse is not None:
68         try:
69             resp = parse(output_format=schema, **base)
70         except TypeError:
71             resp = None # SDK-Variante akzeptiert die Argumente nicht -> Fallback
72         else:
73             parsed = getattr(resp, "parsed_output", None)
74             if parsed is not None:
75                 return parsed
76             raise OfferGenerationError(
77                 "Die KI hat keine verwertbare Antwort geliefert "
78                 "(mögliche Ablehnung oder Token-Limit erreicht).")
79
80
81     # Fallback: create() mit JSON-Schema und manueller Validierung.
82     try:
83         resp = self.client.messages.create(
84             output_config={
85                 "format": {
86                     "type": "json_schema",
87                     "schema": _strictify(schema.model_json_schema()),
88                 }
89             },
90             **base,
91         )
92     except anthropic.APIError as exc:
93         raise OfferGenerationError(f"Fehler beim KI-Aufruf: {exc}") from exc
94
95     text = next(
96         (b.text for b in resp.content if getattr(b, "type", None) == "text"), ""
97     )
98     if not text:
99         raise OfferGenerationError("Die KI hat eine leere Antwort geliefert.")
100     try:
101         return schema.model_validate_json(text)
102     except Exception as exc:
103         raise OfferGenerationError(
104             f"Die KI-Antwort entsprach nicht dem erwarteten Format: {exc}"
105         ) from exc
106
107     def _call(self, **kwargs: Any) -> Any:
108         """Wrappt _structured und übersetzt API-Fehler in klare Meldungen."""
109         try:
110             return self._structured(**kwargs)
111         except anthropic.AuthenticationError as exc:
112             raise OfferGenerationError(
113                 "Anmeldung bei der Claude-API fehlgeschlagen. Bitte ANTHROPIC_API_KEY prüfen."
114             ) from exc
115         except anthropic.APIError as exc:
116             raise OfferGenerationError(f"Fehler beim KI-Aufruf: {exc}") from exc
117
118     # -- Schritt 1: Anfrage analysieren -----
119
120     def analyze_request(self, email_text: str, catalog: Catalog) -> RequestAnalysis:
121         system = (

```

```

122         "Du bist ein erfahrener Vertriebsassistent. Analysiere die E-Mail einer "
123         "Angebotsanfrage und extrahiere strukturiert die Kundendaten und die "
124         "angefragten Positionen.\n"
125         "- Erfinde keine Informationen. Was nicht in der E-Mail steht, bleibt leer "
126         "(leerer String, 0 oder leere Liste).\n"
127         "- Erfasse jede angefragte Position einzeln mit Menge und Einheit, soweit genannt.\n"
128         "- Halte open_questions kurz und nur für echte Unklarheiten."
129     )
130     user = f"Hier ist die Angebotsanfrage des Kunden:\n\n---\n{email_text}\n---"
131     return self._call(system=system, user=user, schema=RequestAnalysis,
132                       thinking=False, max_tokens=8000)
133
134 # -- Schritt 2: Angebot entwerfen -----
135
136 def draft_offer(
137     self, analysis: RequestAnalysis, catalog: Catalog, template_text: str
138 ) -> OfferDraft:
139     system = (
140         "Du bist ein Vertriebsprofi und erstellst ein professionelles, "
141         "verkaufsstarkes Angebot auf Deutsch.\n"
142         "Regeln:\n"
143         "- Wähle Positionen AUSSCHLIESSLICH aus dem bereitgestellten Produktkatalog "
144         "und referenziere sie über die korrekte Artikelnummer (sku).\n"
145         "- Erfinde keine Artikel und KEINE Preise. Die Preise und Summen berechnet "
146         "das System autoritativ aus dem Katalog – gib selbst keine Preise an.\n"
147         "- Wähle die Artikel, deren Spezifikationen am besten zur Anfrage passen.\n"
148         "- Ist ein angefragter Artikel nicht im Katalog, nimm ihn NICHT als Position "
149         "auf, sondern weise im closing_text transparent darauf hin.\n"
150         "- Orientiere dich in Tonalität, Aufbau und Formulierungen an der Beispiel-"
151         "Vorlage, falls eine vorhanden ist.\n"
152         "- Schreibe vollständige, höfliche Texte (Anrede, Bezug zur Anfrage, "
153         "Mehrwert, nächste Schritte)."
154     )
155     template_block = (
156         f"\n\n=== BEISPIEL-VORLAGE (nur Stil/Struktur) ===\n{template_text}"
157         if template_text.strip()
158         else "\n\n(Keine Beispiel-Vorlage vorhanden – nutze einen üblichen, "
159         "professionellen Angebotsaufbau.)"
160     )
161     user = (
162         "=== ANALYSE DER KUNDENANFRAGE (JSON) ===\n"
163         f"{analysis.model_dump_json(indent=2)}\n\n"
164         "=== PRODUKTKATALOG ===\n"
165         f"{catalog.to_prompt_text()}"
166         f"{template_block}\n\n"
167         "Erstelle nun den Angebotsentwurf. Denke an: passende Artikel per sku, "
168         "sinnvolle Mengen, überzeugende Texte, Zahlungs- und Lieferbedingungen, "
169         "Gültigkeitsdauer in Tagen."
170     )
171     return self._call(system=system, user=user, schema=OfferDraft, max_tokens=16000)
172
173 # -- Schritt 3: Selbstprüfung -----
174
175 def review_offer(
176     self,
177     offer_review_text: str,
178     analysis: RequestAnalysis,
179     catalog: Catalog,
180     template_text: str,
181 ) -> ReviewResult:
182     system = (
183         "Du bist eine sorgfältige zweite Person im Vertrieb und prüfst ein bereits "
184         "erstelltes Angebot kritisch gegen die ursprüngliche Anfrage und den Katalog.\n"
185         "Prüfe insbesondere:\n"
186         "1. Wurde alles Angefragte berücksichtigt? Fehlt etwas?\n"
187         "2. Passen die gewählten Artikel und ihre Spezifikationen zur Anfrage?\n"
188         "3. Sind die Mengen plausibel und korrekt?\n"
189         "4. Ist der Text professionell, fehlerfrei, höflich und konsistent zur Vorlage?\n"
190         "Wichtig: Preise und Summen wurden vom System autoritativ aus dem Katalog "
191         "berechnet – prüfe sie NICHT rechnerisch und ändere keine Preise. Weise aber "
192         "auf falsch ausgewählte Artikel hin.\n"

```

```

193         "Liste konkrete Probleme in 'issues'. Setze approved=true nur, wenn keine "
194         "wesentlichen Mängel bestehen. Gib in 'corrected_offer' eine verbesserte "
195         "Fassung zurück (bei approved=true praktisch identisch). Verwende auch dort "
196         "nur Artikelnummern aus dem Katalog und keine Preise."
197     )
198     template_block = (
199         f"\n\n=== BEISPIEL-VORLAGE ===\n{template_text}"
200         if template_text.strip()
201         else ""
202     )
203     user = (
204         "=== URSPRÜNGLICHE ANFRAGE (JSON) ===\n"
205         f"{analysis.model_dump_json(indent=2)}\n\n"
206         "=== AKTUELLES ANGEBOT (inkl. systemberechneter Preise) ===\n"
207         f"{offer_review_text}\n\n"
208         "=== PRODUKTKATALOG ===\n"
209         f"{catalog.to_prompt_text()}"
210         f"{template_block}\n\n"
211         "Prüfe das Angebot und liefere das Ergebnis im geforderten Format."
212     )
213     return self._call(system=system, user=user, schema=ReviewResult, max_tokens=16000)
214

```

catalog.py – Der Produktkatalog (das Warenverzeichnis)

Diese Datei liest die CSV-Datei mit allen Produkten ein – vergleichbar mit einem Warenverzeichnis.

Sie erkennt automatisch, welche Spalte Artikelnummer, Name und Preis enthält (auch bei unterschiedlichen Überschriften), und kann zu jeder Artikelnummer die Details nachschlagen.

Die wichtigsten Teile

- **Catalog.load** — liest die CSV; erkennt Trennzeichen und Spalten automatisch.
- **lookup** — schlägt Preis und Daten zu einer Artikelnummer nach.
- **to_prompt_text** — fasst den Katalog kompakt zusammen, damit die KI ihn »lesen« kann.

Quellcode: `angebots_ki/catalog.py`

```
1  """Laden und Aufbereiten des Produktkatalogs (CSV)."""
2
3  from __future__ import annotations
4
5  import io
6  from typing import Any, Optional
7
8  import pandas as pd
9
10 from .errors import OfferGenerationError
11 from .utils import fmt_money, is_empty, parse_price
12
13 # Mögliche Spaltennamen (klein geschrieben) je Feld – das erste Treffer-Feld gewinnt.
14 _COLUMN_CANDIDATES = {
15     "sku": [
16         "sku", "artikelnummer", "artikel-nr", "artikelnr", "art-nr", "artnr",
17         "art.-nr.", "artikel_nr", "nummer", "id",
18     ],
19     "name": ["name", "bezeichnung", "produkt", "produktname", "artikel", "titel"],
20     "price": [
21         "einzelpreis", "stückpreis", "stueckpreis", "preis", "price", "nettopreis",
22         "netto", "unit_price", "listenpreis", "vk",
23     ],
24     "unit": ["einheit", "unit", "mengeneinheit", "me"],
25     "description": ["beschreibung", "description", "details", "spezifikation", "info"],
26     "currency": ["währung", "waehrung", "currency"],
27 }
28
29
30 def _pick_column(columns: list[str], candidates: list[str]) -> Optional[str]:
31     lower = {c.lower().strip(): c for c in columns}
32     for cand in candidates:
33         if cand in lower:
34             return lower[cand]
35     return None
36
37
38 class Catalog:
39     """Produktkatalog mit tolerantem Spalten-Mapping und Lookup per SKU."""
40
41     def __init__(self, df: pd.DataFrame):
42         cols = list(df.columns)
43         self.col_sku = _pick_column(cols, _COLUMN_CANDIDATES["sku"])
44         self.col_name = _pick_column(cols, _COLUMN_CANDIDATES["name"])
45         self.col_price = _pick_column(cols, _COLUMN_CANDIDATES["price"])
46         self.col_unit = _pick_column(cols, _COLUMN_CANDIDATES["unit"])
47         self.col_desc = _pick_column(cols, _COLUMN_CANDIDATES["description"])
48         self.col_currency = _pick_column(cols, _COLUMN_CANDIDATES["currency"])
49
50         if not self.col_sku or not self.col_name or not self.col_price:
51             raise OfferGenerationError(
52                 "Im Katalog fehlt eine erkennbare Spalte für Artikelnummer, Name "
53                 "oder Preis. Erwartet werden z. B. 'Artikelnummer', 'Name', 'Preis'."
```

```

54         )
55
56     self.df = df.copy()
57     # Alle Spalten, die nicht bereits belegt sind, gelten als Spezifikationen.
58     used = {
59         self.col_sku, self.col_name, self.col_price, self.col_unit,
60         self.col_desc, self.col_currency,
61     }
62     self.spec_cols = [c for c in cols if c not in used]
63
64     # Währung aus den Daten ableiten, sonst EUR.
65     self.currency = "EUR"
66     if self.col_currency:
67         vals = [str(v).strip() for v in self.df[self.col_currency] if not is_empty(v)]
68         if vals:
69             self.currency = vals[0].upper()
70
71     # Schneller Lookup-Index per SKU (als String, getrimmt).
72     self._index: dict[str, int] = {}
73     for pos, value in self.df[self.col_sku].items():
74         if not is_empty(value):
75             self._index[str(value).strip()] = pos
76
77     # -- Laden -----
78
79     @classmethod
80     def load(cls, file_or_path: Any) -> "Catalog":
81         """Lädt eine CSV (Pfad oder dateiähnliches Objekt) mit Trennzeichen-Erkennung."""
82         # Bytes für mehrfaches Lesen puffern (UploadedFile wird sonst leer gelesen).
83         if hasattr(file_or_path, "read"):
84             raw = file_or_path.read()
85             if isinstance(raw, str):
86                 raw = raw.encode("utf-8")
87             buffer: Any = io.BytesIO(raw)
88         else:
89             buffer = file_or_path
90
91         last_error: Optional[Exception] = None
92         for encoding in ("utf-8-sig", "utf-8", "cp1252", "latin-1"):
93             try:
94                 if hasattr(buffer, "seek"):
95                     buffer.seek(0)
96                 # sep=None + engine="python" erkennt , ; oder Tab automatisch.
97                 df = pd.read_csv(buffer, sep=None, engine="python", dtype=str,
98                               encoding=encoding, keep_default_na=False)
99                 if df.shape[1] >= 2:
100                     return cls(df)
101             except Exception as exc: # nächste Kodierung versuchen
102                 last_error = exc
103
104         raise OfferGenerationError(
105             f"Der Produktkatalog konnte nicht gelesen werden: {last_error}"
106         )
107
108     # -- Zugriff -----
109
110     def __len__(self) -> int:
111         return len(self.df)
112
113     def _price_of(self, row: pd.Series) -> float:
114         return round(parse_price(row[self.col_price]), 2)
115
116     def _unit_of(self, row: pd.Series) -> str:
117         if self.col_unit and not is_empty(row[self.col_unit]):
118             return str(row[self.col_unit]).strip()
119         return "Stück"
120
121     def lookup(self, sku: str) -> Optional[dict]:
122         """Liefert die Katalogdaten zu einer SKU oder None."""
123         if is_empty(sku):
124             return None

```



```

125     pos = self._index.get(str(sku).strip())
126     if pos is None:
127         return None
128     row = self.df.loc[pos]
129     specs = {
130         c: str(row[c]).strip()
131         for c in self.spec_cols
132         if not is_empty(row[c])
133     }
134     return {
135         "sku": str(row[self.col_sku]).strip(),
136         "name": str(row[self.col_name]).strip(),
137         "unit_price": self._price_of(row),
138         "unit": self._unit_of(row),
139         "description": "" if not self.col_desc else str(row[self.col_desc]).strip(),
140         "specs": specs,
141     }
142
143     def to_prompt_text(self) -> str:
144         """Kompakte Textdarstellung des gesamten Katalogs für die KI."""
145         lines: list[str] = []
146         for _, row in self.df.iterrows():
147             if is_empty(row[self.col_sku]):
148                 continue
149             parts = [
150                 f"SKU: {str(row[self.col_sku]).strip()}",
151                 f"Name: {str(row[self.col_name]).strip()}",
152                 f"Preis: {fmt_money(self._price_of(row), self.currency)} / {self._unit_of(row)}",
153             ]
154             if self.col_desc and not is_empty(row[self.col_desc]):
155                 parts.append(f"Beschreibung: {str(row[self.col_desc]).strip()}")
156             specs = [
157                 f"{c}={str(row[c]).strip()}"
158                 for c in self.spec_cols
159                 if not is_empty(row[c])
160             ]
161             if specs:
162                 parts.append("Spezifikationen: " + ", ".join(specs))
163             lines.append(" | ".join(parts))
164         return "\n".join(lines)
165
166     def preview_df(self) -> pd.DataFrame:
167         """DataFrame für die Vorschau in der Oberfläche."""
168         return self.df
169

```

storage.py – Der Draht zur Cloud (AWS S3)

Optionaler Zusatz: Statt Katalog und Vorlagen jedes Mal hochzuladen, kann das Programm sie aus einem privaten Online-Speicher (Amazon S3) laden – wie aus einem abschließbaren Aktenschrank im Internet.

Diese Datei kümmert sich um das Anmelden mit Zugangsdaten, das Auflisten der verfügbaren Dateien und das Herunterladen. Ist nichts eingerichtet, wird der Teil einfach übersprungen und die App nutzt weiter Upload/Beispiele.

Die wichtigsten Teile

- **is_configured** — prüft, ob Zugangsdaten und Bucket vorhanden sind (sonst bleibt S3 aus).
- **S3Store.list_files** — listet die verfügbaren Kataloge/Vorlagen im Speicher auf.
- **S3Store.get_bytes** — lädt eine ausgewählte Datei herunter.

Quellcode: `angebots_ki/storage.py`

```
1  """Optionaler Zugriff auf Produktdaten und Vorlagen in einem privaten AWS-S3-Bucket.
2
3  Die Anbindung ist bewusst optional: Ist kein Bucket/keine Zugangsdaten gesetzt (oder
4  boto3 nicht installiert), meldet ``is_configured()`` False und die App nutzt weiter
5  den Datei-Upload bzw. die lokalen Beispieldaten.
6
7  Konfiguration ausschließlich über Umgebungsvariablen (auf Streamlit Community Cloud
8  aus den Secrets gespiegelt):
9
10     AWS_ACCESS_KEY_ID Zugangsschlüssel des IAM-Nutzers
11     AWS_SECRET_ACCESS_KEY geheimer Schlüssel des IAM-Nutzers
12     AWS_DEFAULT_REGION z. B. eu-central-1
13     S3_BUCKET Name des Buckets
14     S3_CATALOG_PREFIX Ordner für Kataloge (Standard: produktdaten/)
15     S3_TEMPLATE_PREFIX Ordner für Vorlagen (Standard: vorlagen/)
16 """
17
18 from __future__ import annotations
19
20 import os
21
22 from .errors import OfferGenerationError
23
24
25 def _config() -> dict:
26     return {
27         "bucket": os.getenv("S3_BUCKET", "").strip(),
28         "region": (os.getenv("AWS_DEFAULT_REGION") or os.getenv("AWS_REGION") or "").strip(),
29         "catalog_prefix": os.getenv("S3_CATALOG_PREFIX", "produktdaten/").strip(),
30         "template_prefix": os.getenv("S3_TEMPLATE_PREFIX", "vorlagen/").strip(),
31     }
32
33
34 def catalog_prefix() -> str:
35     return _config()["catalog_prefix"]
36
37
38 def template_prefix() -> str:
39     return _config()["template_prefix"]
40
41
42 def _boto3_available() -> bool:
43     try:
44         import boto3 # noqa: F401
45         return True
46     except Exception:
47         return False
48
49
50 def is_configured() -> bool:
```

```

51     """True, wenn Bucket + Zugangsdaten vorhanden sind und boto3 installiert ist."""
52     cfg = _config()
53     has_creds = bool(os.getenv("AWS_ACCESS_KEY_ID") and os.getenv("AWS_SECRET_ACCESS_KEY"))
54     return bool(cfg["bucket"]) and has_creds and _boto3_available()
55
56
57 class S3Store:
58     """Dünne Hülle um boto3 für Auflisten und Herunterladen von Objekten."""
59
60     def __init__(self) -> None:
61         try:
62             import boto3
63         except ImportError as exc: # pragma: no cover
64             raise OfferGenerationError(
65                 "Das Paket 'boto3' ist nicht installiert. Bitte 'pip install boto3' ausführen."
66             ) from exc
67
68         cfg = _config()
69         if not cfg["bucket"]:
70             raise OfferGenerationError("Kein S3-Bucket konfiguriert (S3_BUCKET fehlt).")
71
72         self.bucket = cfg["bucket"]
73         client_kwargs = {}
74         if cfg["region"]:
75             client_kwargs["region_name"] = cfg["region"]
76         self.client = boto3.client("s3", **client_kwargs)
77
78     def list_files(self, prefix: str, extensions: tuple = ()) -> list:
79         """Listet Objekt-Schlüssel unter ``prefix`` (optional nach Endung gefiltert)."""
80         keys: list[str] = []
81         try:
82             paginator = self.client.get_paginator("list_objects_v2")
83             for page in paginator.paginate(Bucket=self.bucket, Prefix=prefix):
84                 for obj in page.get("Contents", []):
85                     key = obj["Key"]
86                     if key.endswith("/"):
87                         continue # Ordner-Platzhalter überspringen
88                     if extensions and not key.lower().endswith(tuple(extensions)):
89                         continue
90                     keys.append(key)
91         except Exception as exc:
92             raise OfferGenerationError(
93                 f"AWS S3: Auflisten fehlgeschlagen ({exc}). Bucket-Name, Region und "
94                 f"Zugangsdaten prüfen."
95             ) from exc
96         return sorted(keys)
97
98     def get_bytes(self, key: str) -> bytes:
99         """Lädt den Inhalt eines Objekts als Bytes herunter."""
100         try:
101             resp = self.client.get_object(Bucket=self.bucket, Key=key)
102             return resp["Body"].read()
103         except Exception as exc:
104             raise OfferGenerationError(
105                 f"AWS S3: Datei '{key}' konnte nicht geladen werden ({exc})."
106             ) from exc
107

```

pricing.py – Die Rechenabteilung

Hier werden alle Preise und Summen ausgerechnet – ausschließlich mit den echten Preisen aus dem Katalog, nicht von der KI. So sind die Zahlen immer korrekt.

Auch Mehrwertsteuer und Gesamtsumme entstehen an dieser Stelle.

Die wichtigsten Teile

- **price_offer** — setzt je Position den Katalogpreis ein und rechnet Zwischensumme, MwSt und Gesamt.
- **Warnungen** — weisen darauf hin, wenn ein gewünschter Artikel nicht im Katalog ist.
- **assemble_final** — baut das fertige Angebot mit Nummer, Datum und Gültigkeit zusammen.

Quellcode: `angebots_ki/pricing.py`

```
1  """Preisberechnung (autoritativ aus dem Katalog) und Zusammenbau des Endangebots."""
2
3  from __future__ import annotations
4
5  from .catalog import Catalog
6  from .config import Settings
7  from .models import (
8      FinalOffer,
9      OfferDraft,
10     PricedLineItem,
11     PricedOffer,
12     RequestAnalysis,
13 )
14 from .utils import add_days_str, fmt_money, fmt_qty, make_offer_number, today_str
15
16
17 def price_offer(draft: OfferDraft, catalog: Catalog, settings: Settings) -> PricedOffer:
18     """Setzt für jede Position den Katalogpreis ein und berechnet die Summen.
19
20     Die KI liefert nur SKU + Menge + Texte; Preise kommen ausschließlich hier aus
21     dem Katalog. So sind die Zahlen garantiert konsistent und nicht erfunden.
22     """
23     items: list[PricedLineItem] = []
24     for pos, di in enumerate(draft.line_items, start=1):
25         entry = catalog.lookup(di.sku)
26         warning = None
27         if entry is not None:
28             sku = entry["sku"]
29             name = entry["name"]
30             unit_price = entry["unit_price"]
31             unit = entry["unit"] or di.unit or "Stück"
32             in_catalog = True
33         else:
34             sku = di.sku
35             name = di.name or di.sku or "Position"
36             unit_price = 0.0
37             unit = di.unit or "Stück"
38             in_catalog = False
39             ref = di.sku or di.name or "unbekannt"
40             warning = (
41                 f"Position {pos} ('{ref}') wurde nicht im Katalog gefunden - "
42                 f"Preis bitte manuell ergänzen."
43             )
44
45         quantity = float(di.quantity or 0)
46         line_total = round(quantity * unit_price, 2)
47         items.append(
48             PricedLineItem(
49                 position=pos,
50                 sku=sku,
51                 name=name,
52                 description=di.description,
53                 quantity=quantity,
```

```

54         unit=unit,
55         unit_price=unit_price,
56         line_total=line_total,
57         in_catalog=in_catalog,
58         warning=warning,
59     )
60 )
61
62 subtotal = round(sum(i.line_total for i in items), 2)
63 vat_amount = round(subtotal * settings.vat_rate, 2)
64 total = round(subtotal + vat_amount, 2)
65 return PricedOffer(line_items=items, subtotal=subtotal, vat_amount=vat_amount, total=total)
66
67
68 def render_priced_for_review(
69     draft: OfferDraft,
70     priced: PricedOffer,
71     analysis: RequestAnalysis,
72     settings: Settings,
73 ) -> str:
74     """Lesbare Textfassung des aktuellen Angebots für den Prüfschritt."""
75     lines: list[str] = []
76     lines.append(f"Titel: {draft.offer_title}")
77     lines.append(f"\nEinleitung:\n{draft.intro_text}")
78     lines.append("\nPositionen:")
79     for it in priced.line_items:
80         flag = "" if it.in_catalog else " [NICHT IM KATALOG]"
81         lines.append(
82             f" {it.position}. [{it.sku}] {it.name} - {it.description} | "
83             f"Menge: {fmt_qty(it.quantity)} {it.unit} | "
84             f"Einzelpreis: {fmt_money(it.unit_price, settings.currency)} | "
85             f"Gesamt: {fmt_money(it.line_total, settings.currency)}{flag}"
86         )
87     lines.append(
88         f"\nZwischensumme: {fmt_money(priced.subtotal, settings.currency)}"
89         f"\nMwSt ({settings.vat_rate * 100:.0f}%): {fmt_money(priced.vat_amount, settings.currency)}"
90         f"\nGesamtbetrag: {fmt_money(priced.total, settings.currency)}"
91     )
92     lines.append(f"\nAbschlusstext:\n{draft.closing_text}")
93     lines.append(f"\nZahlungsbedingungen: {draft.payment_terms}")
94     lines.append(f"Lieferbedingungen: {draft.delivery_terms}")
95     lines.append(f"Gültigkeit: {draft.validity_days} Tage")
96     return "\n".join(lines)
97
98
99 def assemble_final(
100     analysis: RequestAnalysis,
101     draft: OfferDraft,
102     priced: PricedOffer,
103     review_issues: list[str],
104     settings: Settings,
105 ) -> FinalOffer:
106     """Baut aus Analyse, Entwurf und berechneten Preisen das finale Angebot."""
107     warnings = [i.warning for i in priced.line_items if i.warning]
108     validity = draft.validity_days or settings.default_validity_days
109     title = draft.offer_title.strip() or "Angebot"
110     return FinalOffer(
111         offer_number=make_offer_number(),
112         date=today_str(),
113         valid_until=add_days_str(validity),
114         customer=analysis.customer,
115         title=title,
116         intro_text=draft.intro_text,
117         line_items=priced.line_items,
118         closing_text=draft.closing_text,
119         payment_terms=draft.payment_terms,
120         delivery_terms=draft.delivery_terms,
121         subtotal=priced.subtotal,
122         vat_rate=settings.vat_rate,
123         vat_amount=priced.vat_amount,
124         total=priced.total,

```

```
125         currency=settings.currency,  
126         warnings=warnings,  
127         review_issues=list(review_issues),  
128     )  
129
```

pdf_builder.py – Der Dokumenten-Designer

Diese Datei verwandelt die fertigen Angebotsdaten in ein schön gestaltetes PDF: mit Briefkopf, Kundenadresse, Positionstabelle, Summen und Fußzeile.

Man kann es sich wie eine Druckvorlage vorstellen, die automatisch ausgefüllt wird.

Die wichtigsten Teile

- **build_offer_pdf** — baut Seite für Seite das PDF zusammen.
- **Positionstabelle** — listet Menge, Einzel- und Gesamtpreis je Position.
- **Fußzeile** — zeigt Firmenkontakt und Seitenzahl.

Quellcode: `angebots_ki/pdf_builder.py`

```
1  """Erzeugt das Angebots-PDF mit ReportLab."""
2
3  from __future__ import annotations
4
5  from functools import partial
6  from io import BytesIO
7
8  from reportlab.lib import colors
9  from reportlab.lib.enums import TA_RIGHT
10 from reportlab.lib.pagesizes import A4
11 from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
12 from reportlab.lib.units import mm
13 from reportlab.platypus import (
14     HRFflowable,
15     Paragraph,
16     SimpleDocTemplate,
17     Spacer,
18     Table,
19     TableStyle,
20 )
21
22 from .config import Settings
23 from .models import FinalOffer
24 from .utils import fmt_money, fmt_qty
25
26 ACCENT = colors.HexColor("#1F3A5F")
27 LIGHT = colors.HexColor("#EEF2F7")
28 GREY = colors.HexColor("#6B7280")
29
30
31 def _styles() -> dict:
32     base = getSampleStyleSheet()
33     return {
34         "small": ParagraphStyle("small", parent=base["Normal"], fontSize=8, leading=10,
35                                 textColor=GREY),
36         "small_r": ParagraphStyle("small_r", parent=base["Normal"], fontSize=8,
37                                   leading=10, textColor=GREY, alignment=TA_RIGHT),
38         "normal": ParagraphStyle("normal", parent=base["Normal"], fontSize=9.5,
39                                   leading=13.5),
40         "addr": ParagraphStyle("addr", parent=base["Normal"], fontSize=10, leading=14),
41         "title": ParagraphStyle("title", parent=base["Heading1"], fontSize=16,
42                                   leading=20, textColor=ACCENT, spaceAfter=2),
43         "h2": ParagraphStyle("h2", parent=base["Heading2"], fontSize=10.5, leading=14,
44                               textColor=ACCENT, spaceBefore=6, spaceAfter=2),
45         "cell": ParagraphStyle("cell", parent=base["Normal"], fontSize=9, leading=12),
46         "cell_name": ParagraphStyle("cell_name", parent=base["Normal"], fontSize=9,
47                                       leading=12, fontName="Helvetica-Bold"),
48         "cell_r": ParagraphStyle("cell_r", parent=base["Normal"], fontSize=9,
49                                   leading=12, alignment=TA_RIGHT),
50         "th": ParagraphStyle("th", parent=base["Normal"], fontSize=9,
51                               textColor=colors.white, fontName="Helvetica-Bold"),
52         "th_r": ParagraphStyle("th_r", parent=base["Normal"], fontSize=9,
53                                 textColor=colors.white, fontName="Helvetica-Bold",
```

```

54         alignment=TA_RIGHT),
55     }
56
57
58 def _paragraphs(text: str, style) -> list:
59     """Wandelt Mehrzeilen-Text in mehrere Paragraphs (Absätze bleiben erhalten)."""
60     out = []
61     for block in (text or "").split("\n"):
62         block = block.strip()
63         out.append(Paragraph(block.replace("&", "&"), style) if block else Spacer(1, 4))
64     return out
65
66
67 def _footer(canvas, doc, company, st) -> None:
68     canvas.saveState()
69     width, _ = A4
70     y = 12 * mm
71     canvas.setStrokeColor(LIGHT)
72     canvas.setLineWidth(0.5)
73     canvas.line(doc.leftMargin, y + 6 * mm, width - doc.rightMargin, y + 6 * mm)
74     canvas.setFont("Helvetica", 7)
75     canvas.setFillColor(GREY)
76     line1 = f"{company.name} · {' · '.join(company.address_lines)}"
77     line2 = f"Tel.: {company.phone} · {company.email} · {company.website} · {company.tax_id}"
78     canvas.drawString(doc.leftMargin, y + 2.5 * mm, line1[:140])
79     canvas.drawString(doc.leftMargin, y, line2[:160])
80     canvas.drawRightString(width - doc.rightMargin, y, f"Seite {doc.page}")
81     canvas.restoreState()
82
83
84 def build_offer_pdf(offer: FinalOffer, settings: Settings) -> bytes:
85     """Baut das fertige Angebots-PDF und gibt es als Bytes zurück."""
86     s = _styles()
87     company = settings.company
88     buf = BytesIO()
89     doc = SimpleDocTemplate(
90         buf, pagesize=A4,
91         leftMargin=18 * mm, rightMargin=18 * mm,
92         topMargin=16 * mm, bottomMargin=24 * mm,
93         title=f"Angebot {offer.offer_number}", author=company.name,
94     )
95
96     story: list = []
97
98     # Briefkopf: Firmenname (Akzent) + Kontakt rechts
99     header = Table(
100         [[
101             Paragraph(f"<b>{company.name}</b>", ParagraphStyle(
102                 "co", fontSize=15, leading=18, textColor=ACCENT)),
103             Paragraph("<br/>".join([*company.address_lines, company.phone,
104                                     company.email, company.website]), s["small_r"]),
105         ]],
106         colWidths=[doc.width * 0.55, doc.width * 0.45],
107     )
108     header.setStyle(TableStyle([
109         ("VALIGN", (0, 0), (-1, -1), "TOP"),
110         ("LEFTPADDING", (0, 0), (-1, -1), 0),
111         ("RIGHTPADDING", (0, 0), (-1, -1), 0),
112     ]))
113     story.append(header)
114     story.append(Spacer(1, 4))
115     story.append(HRFlowable(width="100%", thickness=1.2, color=ACCENT))
116     story.append(Spacer(1, 10))
117
118     # Kundenadresse links + Angebots-Metadaten rechts
119     cust = offer.customer
120     addr_lines = [cust.company] if cust.company else []
121     if cust.contact_name:
122         addr_lines.append(cust.contact_name)
123     if cust.address:
124         addr_lines.extend(cust.address.split("\n"))

```



```

125     if not addr_lines:
126         addr_lines = ["(Kundenanschrift bitte ergänzen)"]
127     addr_html = "<br/>".join(line.strip() for line in addr_lines if line.strip())
128
129     meta = Table(
130         [
131             ["Angebotsnummer:", offer.offer_number],
132             ["Datum:", offer.date],
133             ["Gültig bis:", offer.valid_until],
134             ["Kunden-E-Mail:", cust.email or "-"],
135         ],
136         colWidths=[28 * mm, doc.width * 0.45 - 28 * mm],
137     )
138     meta.setStyle(TableStyle([
139         ("FONTSIZE", (0, 0), (-1, -1), 9),
140         ("TEXTCOLOR", (0, 0), (0, -1), GREY),
141         ("TOPPADDING", (0, 0), (-1, -1), 1),
142         ("BOTTOPPADDING", (0, 0), (-1, -1), 1),
143         ("LEFTPADDING", (0, 0), (-1, -1), 0),
144     ]))
145     addr_block = Table([[Paragraph(addr_html, s["addr"]), meta]],
146                        colWidths=[doc.width * 0.55, doc.width * 0.45])
147     addr_block.setStyle(TableStyle([
148         ("VALIGN", (0, 0), (-1, -1), "TOP"),
149         ("LEFTPADDING", (0, 0), (-1, -1), 0),
150         ("RIGHTPADDING", (0, 0), (-1, -1), 0),
151     ]))
152     story.append(addr_block)
153     story.append(Spacer(1, 14))
154
155     # Titel + Einleitung
156     story.append(Paragraph(offer.title.replace("&", "&"), s["title"]))
157     story.append(Spacer(1, 6))
158     story.extend(_paragraphs(offer.intro_text, s["normal"]))
159     story.append(Spacer(1, 10))
160
161     # Positionstabelle
162     head = [
163         Paragraph("Pos.", s["th"]), Paragraph("Art.-Nr.", s["th"]),
164         Paragraph("Beschreibung", s["th"]), Paragraph("Menge", s["th_r"]),
165         Paragraph("Einzelpreis", s["th_r"]), Paragraph("Gesamt", s["th_r"]),
166     ]
167     rows = [head]
168     for it in offer.line_items:
169         desc = f"<b>{it.name}</b>".replace("&", "&")
170         if it.description:
171             desc += "<br/>" + it.description.replace("&", "&")
172         rows.append([
173             Paragraph(str(it.position), s["cell"]),
174             Paragraph(it.sku or "-", s["cell"]),
175             Paragraph(desc, s["cell"]),
176             Paragraph(f"{fmt_qty(it.quantity)} {it.unit}", s["cell_r"]),
177             Paragraph(fmt_money(it.unit_price, offer.currency), s["cell_r"]),
178             Paragraph(fmt_money(it.line_total, offer.currency), s["cell_r"]),
179         ])
180
181     table = Table(
182         rows, repeatRows=1,
183         colWidths=[12 * mm, 24 * mm, doc.width - 12 * mm - 24 * mm - 18 * mm - 24 * mm - 24 * mm,
184                   18 * mm, 24 * mm, 24 * mm],
185     )
186     table.setStyle(TableStyle([
187         ("BACKGROUND", (0, 0), (-1, 0), ACCENT),
188         ("VALIGN", (0, 0), (-1, -1), "TOP"),
189         ("TOPPADDING", (0, 0), (-1, -1), 5),
190         ("BOTTOPPADDING", (0, 0), (-1, -1), 5),
191         ("LEFTPADDING", (0, 0), (-1, -1), 5),
192         ("RIGHTPADDING", (0, 0), (-1, -1), 5),
193         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, LIGHT]),
194         ("LINEBELOW", (0, 1), (-1, -1), 0.25, colors.HexColor("#D1D5DB")),
195     ]))

```

```

196 story.append(table)
197 story.append(Spacer(1, 8))
198
199 # Summenblock (rechtsbündig)
200 summary = Table(
201     [
202         ["Zwischensumme (netto):", fmt_money(offer.subtotal, offer.currency)],
203         [f"zzgl. MwSt ({offer.vat_rate * 100:.0f}%):",
204          fmt_money(offer.vat_amount, offer.currency)],
205         ["Gesamtbetrag:", fmt_money(offer.total, offer.currency)],
206     ],
207     colWidths=[45 * mm, 35 * mm],
208     hAlign="RIGHT",
209 )
210 summary.setStyle(TableStyle([
211     ("FONTSIZE", (0, 0), (-1, -1), 9.5),
212     ("ALIGN", (0, 0), (-1, -1), "RIGHT"),
213     ("TOPPADDING", (0, 0), (-1, -1), 2),
214     ("BOTTOMPADDING", (0, 0), (-1, -1), 2),
215     ("LINEABOVE", (0, 2), (-1, 2), 0.75, ACCENT),
216     ("FONTNAME", (0, 2), (-1, 2), "Helvetica-Bold"),
217     ("TEXTCOLOR", (0, 2), (-1, 2), ACCENT),
218     ("TOPPADDING", (0, 2), (-1, 2), 4),
219 ]))
220 story.append(summary)
221 story.append(Spacer(1, 12))
222
223 # Abschlusstext
224 if offer.closing_text.strip():
225     story.extend(_paragraphs(offer.closing_text, s["normal"]))
226     story.append(Spacer(1, 8))
227
228 # Konditionen
229 story.append(Paragraph("Konditionen", s["h2"]))
230 cond_bits = []
231 if offer.payment_terms.strip():
232     cond_bits.append(f"<b>Zahlung:</b> {offer.payment_terms}")
233 if offer.delivery_terms.strip():
234     cond_bits.append(f"<b>Lieferung/Leistung:</b> {offer.delivery_terms}")
235 cond_bits.append(f"<b>Gültigkeit:</b> Dieses Angebot ist gültig bis {offer.valid_until}.")
236 for bit in cond_bits:
237     story.append(Paragraph(bit.replace("&", "&amp;"), s["normal"]))
238
239 story.append(Spacer(1, 14))
240 story.append(Paragraph(
241     f"Wir freuen uns auf Ihre Rückmeldung.<br/>Mit freundlichen Grüßen<br/><br/>"
242     f"{company.name}", s["normal"]))
243
244 on_page = partial(_footer, company=company, st=s)
245 doc.build(story, onFirstPage=on_page, onLaterPages=on_page)
246 return buf.getvalue()
247

```

models.py – Die Datenschablonen (Formulare)

Damit sich die Programmteile »verstehen«, gibt es feste Schablonen, wie Daten aussehen müssen – ähnlich einem Formular mit festen Feldern.

Hier ist z. B. festgelegt, dass eine Position eine Artikelnummer, eine Menge und einen Preis hat.

Die wichtigsten Teile

- **RequestAnalysis** — das Ergebnis der E-Mail-Analyse (Kunde + Positionen).
- **OfferDraft** — der Angebotsentwurf der KI (noch ohne Preise).
- **ReviewResult** — das Ergebnis der Selbstprüfung.
- **PricedLineItem / FinalOffer** — das fertige, mit Preisen versehene Angebot.

Quellcode: `angebots_ki/models.py`

```
1  """Datenmodelle für KI-Ein-/Ausgabe (Pydantic) und interne Strukturen (Dataclasses).
2
3  Hinweis zu strukturierten Outputs: Alle Pydantic-Felder sind verpflichtend (keine
4  Optionals). Unbekannte Werte trägt die KI als leeren String, 0 oder leere Liste ein.
5  Das hält das JSON-Schema einfach und kompatibel zu den strukturierten Outputs.
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import dataclass, field
11 from typing import Optional
12
13 from pydantic import BaseModel, Field
14
15 # -----
16 # 1) Analyse der Kundenanfrage
17 # -----
18
19
20 class CustomerInfo(BaseModel):
21     company: str = Field(description="Firmenname des Kunden; leer, wenn unbekannt")
22     contact_name: str = Field(description="Ansprechpartner/in; leer, wenn unbekannt")
23     email: str = Field(description="E-Mail-Adresse; leer, wenn unbekannt")
24     phone: str = Field(description="Telefonnummer; leer, wenn unbekannt")
25     address: str = Field(
26         description="Vollständige Anschrift, Zeilen mit \\n getrennt; leer, wenn unbekannt"
27     )
28
29
30 class RequestedItem(BaseModel):
31     description: str = Field(description="Was der Kunde angefragt hat")
32     quantity: float = Field(description="Angefragte Menge; 0, wenn nicht genannt")
33     unit: str = Field(description="Einheit, z. B. 'Stück', 'Stunden'; leer, wenn unklar")
34     notes: str = Field(description="Zusätzliche Hinweise/Anforderungen; leer, wenn keine")
35
36
37 class RequestAnalysis(BaseModel):
38     customer: CustomerInfo
39     items: list[RequestedItem]
40     desired_deadline: str = Field(description="Gewünschter Liefer-/Antworttermin; leer, wenn keiner")
41     summary: str = Field(description="Kurze Zusammenfassung der Anfrage in 1-2 Sätzen")
42     open_questions: list[str] = Field(
43         description="Unklarheiten/offene Punkte, die ein Vertriebler klären würde"
44     )
45
46
47 # -----
48 # 2) Angebotsentwurf (Text + Artikelauswahl - OHNE Preise)
49 # -----
50
51
```

```

52 class DraftLineItem(BaseModel):
53     sku: str = Field(
54         description="Artikelnummer (SKU) aus dem Katalog. Leer lassen, wenn kein "
55         "passender Katalogartikel existiert."
56     )
57     name: str = Field(
58         description="Produktname. Bei Katalogartikeln der Katalogname, sonst Freitext."
59     )
60     description: str = Field(description="Auf den Kunden zugeschnittene Positionsbeschreibung")
61     quantity: float = Field(description="Menge für diese Position")
62     unit: str = Field(description="Einheit, z. B. 'Stück'")
63
64
65 class OfferDraft(BaseModel):
66     offer_title: str = Field(description="Titel/Betreff des Angebots")
67     intro_text: str = Field(description="Einleitender Text (Anrede + Bezug zur Anfrage)")
68     line_items: list[DraftLineItem]
69     closing_text: str = Field(
70         description="Abschlusstext (Mehrwert, nächste Schritte, Hinweise zu nicht "
71         "lieferbaren Wünschen)"
72     )
73     payment_terms: str = Field(description="Zahlungsbedingungen")
74     delivery_terms: str = Field(description="Liefer-/Leistungsbedingungen")
75     validity_days: int = Field(description="Gültigkeit des Angebots in Tagen, z. B. 30")
76
77
78 # -----
79 # 3) Selbstprüfung
80 # -----
81
82
83 class ReviewResult(BaseModel):
84     approved: bool = Field(description="True, wenn keine wesentlichen Probleme bestehen")
85     issues: list[str] = Field(description="Gefundene Probleme/Hinweise; leer, wenn keine")
86     corrected_offer: OfferDraft = Field(
87         description="Verbesserte Fassung des Angebots (bei approved=true nahezu identisch)"
88     )
89
90
91 # -----
92 # Interne Strukturen mit berechneten Preisen (autoritativ aus dem Katalog)
93 # -----
94
95
96 @dataclass
97 class PricedLineItem:
98     position: int
99     sku: str
100     name: str
101     description: str
102     quantity: float
103     unit: str
104     unit_price: float
105     line_total: float
106     in_catalog: bool
107     warning: Optional[str] = None
108
109
110 @dataclass
111 class PricedOffer:
112     line_items: list[PricedLineItem]
113     subtotal: float
114     vat_amount: float
115     total: float
116
117
118 @dataclass
119 class FinalOffer:
120     offer_number: str
121     date: str
122     valid_until: str

```

```
123     customer: CustomerInfo
124     title: str
125     intro_text: str
126     line_items: list[PricedLineItem]
127     closing_text: str
128     payment_terms: str
129     delivery_terms: str
130     subtotal: float
131     vat_rate: float
132     vat_amount: float
133     total: float
134     currency: str
135     warnings: list[str] = field(default_factory=list)
136     review_issues: list[str] = field(default_factory=list)
137
```

config.py – Die Einstellungen

Zentrale Stellschrauben: welches KI-Modell benutzt wird, welcher Mehrwertsteuersatz gilt, wie lange ein Angebot gültig ist und welche Firmendaten im Briefkopf stehen.

Die wichtigsten Teile

- **Settings** — allgemeine Einstellungen (Modell, Steuer, Gültigkeit ...).
- **CompanyInfo** — die eigenen Firmendaten für den Briefkopf.

Quellcode: `angebots_ki/config.py`

```
1  """Konfiguration: Modell, Firmendaten, Steuersatz usw."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass, field
6
7  # Aktuelles, leistungsfähigstes Claude-Modell (Stand der claude-api-Referenz).
8  DEFAULT_MODEL = "claude-opus-4-8"
9
10
11 @dataclass
12 class CompanyInfo:
13     """Daten des eigenen Unternehmens, die im Angebot erscheinen."""
14
15     name: str = "Muster GmbH"
16     address_lines: list[str] = field(
17         default_factory=lambda: ["Musterstraße 1", "12345 Musterstadt", "Deutschland"]
18     )
19     phone: str = "+49 30 1234567"
20     email: str = "vertrieb@muster-gmbh.de"
21     website: str = "www.muster-gmbh.de"
22     tax_id: str = "USt-IdNr. DE123456789"
23     bank: str = "Muster Bank · IBAN DE00 0000 0000 0000 0000 00 · BIC XXXXDEXX"
24
25     def address_block(self) -> str:
26         """Mehrzeiliger Adressblock für den Briefkopf."""
27         return "\n".join([self.name, *self.address_lines])
28
29
30 @dataclass
31 class Settings:
32     """Allgemeine Einstellungen für einen Angebotslauf."""
33
34     model: str = DEFAULT_MODEL
35     currency: str = "EUR"
36     vat_rate: float = 0.19
37     default_validity_days: int = 30
38     # Wie oft die KI höchstens eine Korrekturschleife dreht (mind. 1 Prüflauf).
39     max_review_iterations: int = 1
40     # Adaptive Thinking für die anspruchsvollen Schritte (Entwurf/Prüfung).
41     use_thinking: bool = True
42     company: CompanyInfo = field(default_factory=CompanyInfo)
43
```

utils.py – Der Werkzeugkasten (kleine Helfer)

Eine Sammlung kleiner Hilfsfunktionen, die überall gebraucht werden.

Die wichtigsten Teile

- **fmt_money / fmt_qty** — schöne Darstellung von Beträgen (1.234,56 €) und Mengen.
- **parse_price** — versteht »1.299,00« genauso wie »1299.00«.
- **extract_pdf_text** — liest den Text aus der hochgeladenen Beispiel-Vorlage.

Quellcode: [angebots_ki/utils.py](#)

```
1  """Hilfsfunktionen: Formatierung, Datumswerte, PDF-Textextraktion."""
2
3  from __future__ import annotations
4
5  import math
6  import random
7  import re
8  from datetime import date, timedelta
9  from typing import Any
10
11  _CURRENCY_SYMBOLS = {"EUR": "€", "USD": "$", "GBP": "£", "CHF": "CHF"}
12
13
14  def is_empty(value: Any) -> bool:
15      """True für None, NaN oder leere/whitespace Strings."""
16      if value is None:
17          return True
18      if isinstance(value, float):
19          try:
20              return math.isnan(value)
21          except (TypeError, ValueError):
22              return False
23      return str(value).strip() == ""
24
25
26  def fmt_money(value: float, currency: str = "EUR") -> str:
27      """Formatiert einen Betrag im deutschen Format, z. B. `1.234,56 €`."""
28      try:
29          value = float(value)
30      except (TypeError, ValueError):
31          value = 0.0
32      # 1234567.89 -> "1,234,567.89" -> deutsches Format
33      s = f"{value:,.2f}".replace(",", " ").replace(".", ",").replace(" ", ".")
34      symbol = _CURRENCY_SYMBOLS.get(currency.upper(), currency)
35      return f"{s} {symbol}"
36
37
38  def fmt_qty(value: float) -> str:
39      """Menge ohne unnötige Nachkommastellen (3 statt 3,00)."""
40      try:
41          value = float(value)
42      except (TypeError, ValueError):
43          return "0"
44      if value == int(value):
45          return str(int(value))
46      return f"{value:.2f}".replace(".", ",")
47
48
49  def parse_price(value: Any) -> float:
50      """Wandelt einen Preis-String robust in eine Zahl um.
51
52      Versteht `1.234,56`, `1,234.56`, `1234.56`, `€ 99,90` usw.
53      """
54      if value is None:
55          return 0.0
56      if isinstance(value, (int, float)):
```

```

57         try:
58             return 0.0 if math.isnan(float(value)) else float(value)
59         except (TypeError, ValueError):
60             return 0.0
61
62     s = re.sub(r"^[^0-9.-]", "", str(value))
63     if not s:
64         return 0.0
65
66     has_comma, has_dot = ",", " " in s, "." in s
67     if has_comma and has_dot:
68         # Das letzte Trennzeichen ist das Dezimaltrennzeichen.
69         if s.rfind(",") > s.rfind("."):
70             s = s.replace(".", "").replace(",", ".")
71         else:
72             s = s.replace(",", ".")
73     elif has_comma:
74         # Komma ist Dezimaltrenner, wenn 1-2 Stellen folgen, sonst Tausender.
75         if re.search(r"\d{1,2}$", s):
76             s = s.replace(".", "").replace(",", ".")
77         else:
78             s = s.replace(",", ".")
79     try:
80         return float(s)
81     except ValueError:
82         return 0.0
83
84
85 def today_str() -> str:
86     return date.today().strftime("%d.%m.%Y")
87
88
89 def add_days_str(days: int) -> str:
90     return (date.today() + timedelta(days=int(days or 0))).strftime("%d.%m.%Y")
91
92
93 def make_offer_number() -> str:
94     """Eindeutige Angebotsnummer, z. B. ``ANG-20260616-4821``."""
95     return f"ANG-{date.today():%Y%m%d}-{random.randint(1000, 9999)}"
96
97
98 def extract_pdf_text(file_or_path: Any, max_chars: int = 12000) -> str:
99     """Liest den Text aus einem (Beispiel-)PDF aus.
100
101     ``file_or_path`` kann ein Pfad oder ein dateiähnliches Objekt sein.
102     Bei Problemen wird ein leerer String zurückgegeben – die Vorlage ist optional.
103     """
104     try:
105         from pypdf import PdfReader
106     except ImportError: # pragma: no cover
107         return ""
108
109     try:
110         reader = PdfReader(file_or_path)
111         text = "\n".join(page.extract_text() or "" for page in reader.pages)
112     except Exception:
113         return ""
114
115     text = text.strip()
116     if len(text) > max_chars:
117         text = text[:max_chars] + "\n...(gekürzt)..."
118     return text
119

```


errors.py – Die Fehlermeldungen

Eine winzige, aber wichtige Datei: Sie definiert einen eigenen Fehlertyp, damit Probleme (z. B. »Katalog nicht lesbar«) als klare, verständliche Meldung in der Oberfläche erscheinen statt als technisches Kauderwelsch.

Quellcode: `angebots_ki/errors.py`

```
1  """Fehlerklassen der Angebots-KI."""
2
3  from __future__ import annotations
4
5
6  class OfferGenerationError(Exception):
7      """Wird ausgelöst, wenn die Angebotserstellung fehlschlägt.
8
9      Die Nachricht ist für die Anzeige in der Oberfläche gedacht.
10     """
11
```

__init__.py – Das »Türschild« des Programmordners

Diese Datei macht den Ordner `angebots_ki` zu einem zusammengehörigen »Paket« und stellt die wichtigsten Bausteine zum einfachen Verwenden bereit – eine Art Inhaltsverzeichnis des Programmordners.

Quellcode: `angebots_ki/__init__.py`

```
1  """Angebots-KI - KI-gestützte Erstellung von Angebotsdokumenten als PDF.
2
3  Das Paket liest eine Kunden-E-Mail (Angebotsanfrage), nutzt einen Produktkatalog
4  (CSV) und ein optionales Beispiel-Angebot (PDF) als Vorlage, lässt Claude ein
5  Angebot erstellen, dieses noch einmal selbst prüfen und gibt am Ende ein fertiges
6  PDF aus.
7
8  Wichtig: Preise und Summen stammen IMMER aus dem Katalog und werden in Python
9  berechnet - die KI wählt nur Artikel aus und formuliert die Texte.
10 """
11
12 from .config import CompanyInfo, Settings
13 from .errors import OfferGenerationError
14
15 __all__ = ["CompanyInfo", "Settings", "OfferGenerationError"]
16
```

create_sample_template.py – Hilfsskript: Beispiel-Vorlage

Ein kleines Zusatzprogramm, das ein Beispiel-Angebot als PDF erstellt. Damit kann man die »Vorlage hochladen«-Funktion ausprobieren, ohne selbst ein PDF zu haben.

Quellcode: `scripts/create_sample_template.py`

```

1 """Erzeugt ein Beispiel-Angebot als PDF, das als Vorlage hochgeladen werden kann.
2
3 Aufruf: python scripts/create_sample_template.py
4 Ergebnis: sample_data/beispiel_angebot_vorlage.pdf
5 """
6
7 from __future__ import annotations
8
9 from pathlib import Path
10
11 from reportlab.lib import colors
12 from reportlab.lib.pagesizes import A4
13 from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
14 from reportlab.lib.units import mm
15 from reportlab.platypus import (
16     HFlowable,
17     Paragraph,
18     SimpleDocTemplate,
19     Spacer,
20     Table,
21     TableStyle,
22 )
23
24 OUT = Path(__file__).resolve().parent.parent / "sample_data" / "beispiel_angebot_vorlage.pdf"
25 ACCENT = colors.HexColor("#1F3A5F")
26
27
28 def main() -> None:
29     base = getSampleStyleSheet()
30     normal = ParagraphStyle("n", parent=base["Normal"], fontSize=10, leading=14)
31     title = ParagraphStyle("t", parent=base["Heading1"], fontSize=16, textColor=ACCENT)
32     h2 = ParagraphStyle("h2", parent=base["Heading2"], fontSize=11, textColor=ACCENT)
33
34     OUT.parent.mkdir(parents=True, exist_ok=True)
35     doc = SimpleDocTemplate(str(OUT), pagesize=A4, topMargin=20 * mm, bottomMargin=20 * mm)
36     story = []
37
38     story.append(Paragraph("<b>Muster GmbH</b> · Musterstraße 1 · 12345 Musterstadt", normal))
39     story.append(Spacer(1, 4))
40     story.append(HFlowable(width="100%", color=ACCENT, thickness=1.2))
41     story.append(Spacer(1, 12))
42
43     story.append(Paragraph("Beispiel Kunde GmbH<br/>Beispielweg 5<br/>10115 Berlin", normal))
44     story.append(Spacer(1, 12))
45     story.append(Paragraph("Angebot Nr. ANG-20260101-1000 &nbsp;&nbsp;&nbsp; Datum: 01.01.2026 "&nbsp;&nbsp;&nbsp; Gültig bis: 31.01.2026", normal))
46     story.append(Spacer(1, 12))
47
48     story.append(Paragraph("Angebot über Büroausstattung", title))
49     story.append(Spacer(1, 6))
50     story.append(Paragraph(
51         "Sehr geehrte Frau Beispiel,<br/><br/>vielen Dank für Ihre Anfrage und Ihr "&nbsp;"
52         "Interesse an unseren Produkten. Gerne unterbreiten wir Ihnen das folgende "&nbsp;"
53         "Angebot, das genau auf Ihren Bedarf zugeschnitten ist.", normal))
54     story.append(Spacer(1, 10))
55
56     rows = [
57         ["Pos.", "Art.-Nr.", "Beschreibung", "Menge", "Einzelpreis", "Gesamt"],
58         ["1", "MON-2400", "Office Monitor 24", "2 Stück", "179,00 €", "358,00 €"],
59         ["2", "KB-MX1", "Funkastatur Comfort", "2 Stück", "69,90 €", "139,80 €"],
60     ]

```

```

62 table = Table(rows, colWidths=[12 * mm, 24 * mm, 60 * mm, 22 * mm, 26 * mm, 26 * mm])
63 table.setStyle(TableStyle([
64     ("BACKGROUND", (0, 0), (-1, 0), ACCENT),
65     ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
66     ("FONTSIZE", (0, 0), (-1, -1), 9),
67     ("ALIGN", (3, 0), (-1, -1), "RIGHT"),
68     ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#EEF2F7")]),
69     ("TOPPADDING", (0, 0), (-1, -1), 5),
70     ("BOTTOMPADDING", (0, 0), (-1, -1), 5),
71 ]))
72 story.append(table)
73 story.append(Spacer(1, 8))
74 story.append(Paragraph("Zwischensumme: 497,80 €<br/>zzgl. MwSt (19%): 94,58 €<br/>"
75     "<b>Gesamtbetrag: 592,38 €</b>", normal))
76 story.append(Spacer(1, 12))
77
78 story.append(Paragraph(
79     "Wir sind überzeugt, dass diese Lösung Ihre Anforderungen optimal erfüllt, und "
80     "stehen für Rückfragen jederzeit gern zur Verfügung.", normal))
81 story.append(Spacer(1, 10))
82 story.append(Paragraph("Konditionen", h2))
83 story.append(Paragraph(
84     "<b>Zahlung:</b> 14 Tage netto nach Rechnungserhalt.<br/>"
85     "<b>Lieferung:</b> frei Haus innerhalb von 5 Werktagen.<br/>"
86     "<b>Gültigkeit:</b> Dieses Angebot ist gültig bis 31.01.2026.", normal))
87 story.append(Spacer(1, 14))
88 story.append(Paragraph("Mit freundlichen Grüßen<br/><br/>Muster GmbH", normal))
89
90 doc.build(story)
91 print(f"Beispiel-Vorlage erstellt: {OUT}")
92
93
94 if __name__ == "__main__":
95     main()
96

```

upload_to_s3.py – Hilfsskript: Daten in die Cloud laden

Ein kleines Zusatzprogramm, das die Beispieldaten einmalig in den AWS-Speicher hochlädt – praktisch für die Ersteinrichtung, damit im Online-Speicher gleich ein Katalog (und eine Vorlage) liegen.

Quellcode: scripts/upload_to_s3.py

```
1  """Lädt die lokalen Beispieldaten in den S3-Bucket hoch (einmalige Erstbefüllung).
2
3  Voraussetzung: Umgebungsvariablen AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY,
4  AWS_DEFAULT_REGION und S3_BUCKET sind gesetzt (z. B. über eine .env-Datei).
5
6  Aufruf: python scripts/upload_to_s3.py
7
8  Lädt hoch:
9      sample_data/produktkatalog.csv -> <S3_CATALOG_PREFIX>produktkatalog.csv
10     sample_data/beispiel_angebot_vorlage.pdf -> <S3_TEMPLATE_PREFIX>standard_angebot.pdf (falls vorhanden)
11 """
12
13 from __future__ import annotations
14
15 import os
16 from pathlib import Path
17
18 from dotenv import load_dotenv
19
20 load_dotenv()
21
22 ROOT = Path(__file__).resolve().parent.parent
23 SAMPLE = ROOT / "sample_data"
24
25 CATALOG_PREFIX = os.getenv("S3_CATALOG_PREFIX", "produktkatalog/")
26 TEMPLATE_PREFIX = os.getenv("S3_TEMPLATE_PREFIX", "vorlagen/")
27
28
29 def main() -> None:
30     bucket = os.getenv("S3_BUCKET", "").strip()
31     if not bucket:
32         raise SystemExit("Fehler: S3_BUCKET ist nicht gesetzt (z. B. in .env).")
33     if not (os.getenv("AWS_ACCESS_KEY_ID") and os.getenv("AWS_SECRET_ACCESS_KEY")):
34         raise SystemExit("Fehler: AWS-Zugangsdaten fehlen (AWS_ACCESS_KEY_ID/SECRET).")
35
36     try:
37         import boto3
38     except ImportError:
39         raise SystemExit("Fehler: boto3 nicht installiert. Bitte 'pip install boto3'.")
40
41     region = os.getenv("AWS_DEFAULT_REGION") or os.getenv("AWS_REGION")
42     client = boto3.client("s3", **({"region_name": region} if region else {}))
43
44     uploads = [
45         (SAMPLE / "produktkatalog.csv", CATALOG_PREFIX + "produktkatalog.csv", "text/csv"),
46     ]
47     template = SAMPLE / "beispiel_angebot_vorlage.pdf"
48     if template.exists():
49         uploads.append((template, TEMPLATE_PREFIX + "standard_angebot.pdf", "application/pdf"))
50
51     for path, key, content_type in uploads:
52         if not path.exists():
53             print(f"Übersprungen (nicht gefunden): {path.name}")
54             continue
55         client.upload_file(
56             str(path), bucket, key,
57             ExtraArgs={"ContentType": content_type},
58         )
59         print(f"Hochgeladen: {path.name} -> s3://{bucket}/{key}")
60
61     print("\nFertig. Tipp: Weitere Kataloge/Vorlagen kannst du direkt in der "
```

```
62         "AWS-Konsole in die Ordner "  
63         f"'{CATALOG_PREFIX}' bzw. '{TEMPLATE_PREFIX}' hochladen.")  
64  
65  
66 if __name__ == "__main__":  
67     main()  
68
```